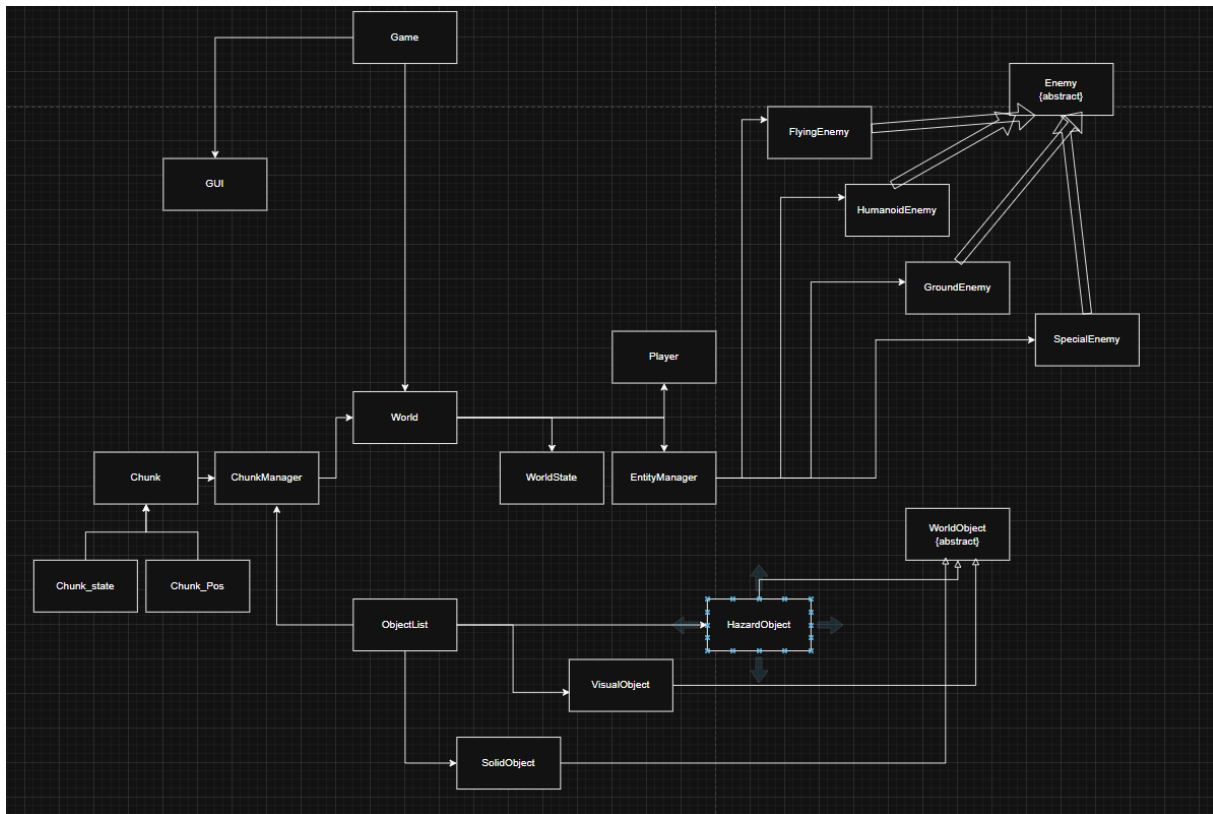


Exposé Informatik-Projekt

Am 03.05.2026 habe ich mithilfe meines Projektmitglieds eine grobe Idee und ein Konzept unseres Spiels erarbeitet. Für die Grundidee haben wir uns für einen 2D-Plattformer entschieden. Grundsätzlich ist es ein skillbasierter Open-World-Plattformer mit Metroidvania¹-Progression und Fokus auf Movement statt Stats. Die Story nimmt Inspiration von einem LitRPG²-Buch namens „Defiance of the Fall“. Im Folgenden ist ein vereinfachtes Klassendiagramm, das mithilfe von ChatGPT überarbeitet und erweitert wurde und voraussichtlich diese Klassen beinhaltet:

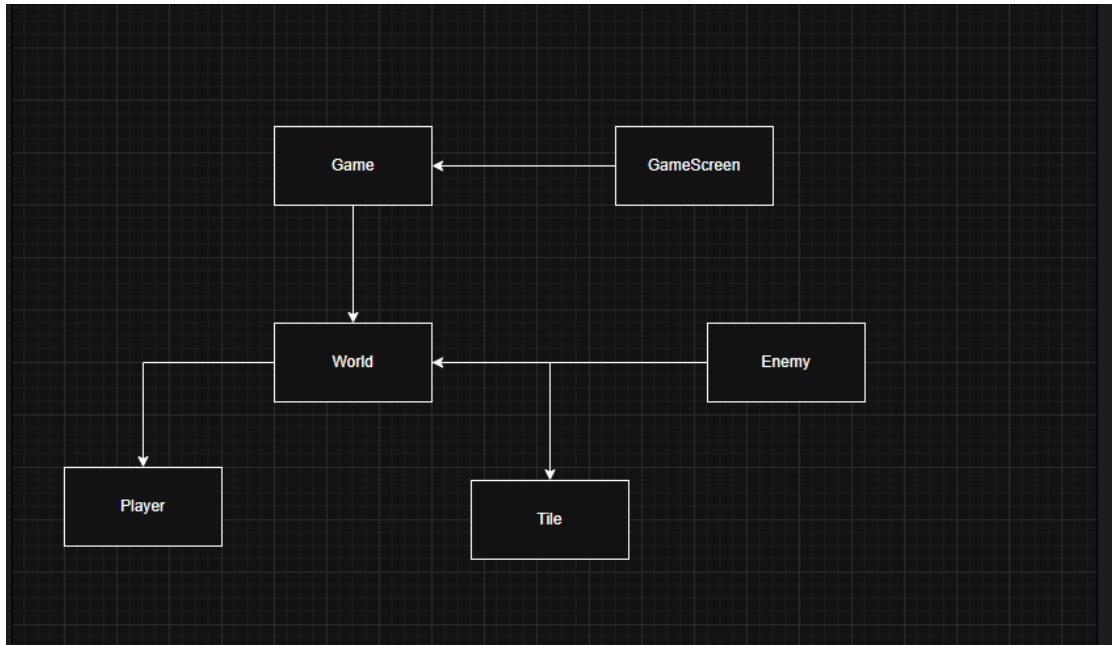


Am 06.05.2026 wurde mir klar, dass das obige Klassendiagramm eher für ein langfristiges Projekt gedacht ist und deshalb stark reduziert wurde. Das Projekt kann in Zukunft mit mehr Zeit auf das obige Klassendiagramm umgebaut werden. Außerdem habe ich die benötigten Klassen und Bibliotheken in das Projekt eingefügt. Zuletzt erstellte ich ein Repository auf GitHub, um das Arbeiten in der Gruppe einfacher zu gestalten. Anschließend habe ich unser Projekt-Board mit Aufgaben befüllt. Das neue Klassendiagramm sieht folgendermaßen aus:

¹ Metroidvania ist ein Computerspielgenre und ein Kofferwort aus den Spielen *Metroid* und *Castlevania*.

Deren Game-Design zeichnet sich durch eine große und zusammenhängende Umgebung aus, durch die der Spieler ohne klare Zielvorgabe navigiert. Durch schrittweise erworbene Fähigkeiten oder Gegenstände können zuvor unzugängliche Bereiche der Spielwelt erschlossen werden.

² LitRPG-Bücher verbinden Fantasy- oder Science-Fiction-Geschichten mit Videospielmechaniken (Stats, Level-Ups, Quests).



Am 08.05.2026 habe ich die Tile-Klasse und die Enemy-Klasse von Vincent überarbeitet und mit der World-Klasse verbunden. Außerdem habe ich in der Player-Klasse die benötigten Attribute sowie ein simples Movement-/Inputsystem hinzugefügt. Meine erste Idee für das Movementsystem war es, die Position bei einem Tastendruck direkt zu ändern. Bei weiterer Recherche fand ich jedoch ein besseres System. Dabei nutzte ich Vektoren zur Berechnung der Geschwindigkeit und der Veränderung der Position bei einem Tastendruck mithilfe der physikalischen Formeln $v = a \cdot t$, $x = v \cdot t$ und $a = a_0$. Zudem habe ich ein Zustandssystem implementiert, das langfristig für kompliziertere Features wie eine Vorwärtsrolle verwendet werden soll.

Am 13.05.2026 habe ich angefangen, das Rendering-System zu implementieren. Die ersten Probleme traten nach dem Implementieren eines AssetManagers auf, der die jeweiligen Texturen lädt. Das System zwischen der Screen-Klasse und der Game-Klasse zu kombinieren, habe ich nicht geschafft. Im Internet habe ich dazu recherchiert und stieß auf einen Beitrag auf StackOverflow³, bei dem eine Implementierung mit einer externen Asset-Klasse gezeigt wurde. Diese habe ich zum Teil angepasst und genutzt.

Das Rendern des Spielers führte mich in eine stundenlange Debug-Suche, bei der ich viele Probleme mit meinem vorherigen Game-World-Screen-System ausbessern musste. Die Probleme und teilweise auch die Lösungen habe ich mithilfe von ChatGPT gefunden und jeweils gelöst. Als der Spieler sich nach der erfolgreichen Anzeige auf dem Bildschirm nicht bewegte, dachte ich, es gäbe einen Fehler in der Bewegungslogik, und habe diese

³ <https://stackoverflow.com/questions/33651323/libgdx-best-way-to-load-all-assets-in-the-game>

deswegen angepasst und verbessert. Es stellte sich aber heraus, dass die Kamera sich mit jeder Bewegung perfekt mitbewegte und es schien, als würde sich nichts bewegen.

Daraufhin habe ich die Kamera dem Spieler mit einer Verzögerung folgen lassen, indem ich einen Offset der Kameraposition verwendet habe. Damit die TileMap auch eingebunden ist, habe ich die TiledMap-Klasse verwendet und eine .tmx-TileMap mit einem Editor erstellt, die als Prototyp dienen soll. Die TileMap wird ganz einfach mithilfe einer TileMapRenderer-Klasse auf dem Bildschirm angezeigt. Die TiledMap-Klasse ersetzt eine eigene Tile-Klasse, da dort bereits alles implementiert ist.

Am 14.05.2026 habe ich ein Hintergrundbild hinzugefügt, das ich online gefunden habe, und einen Effekt namens Parallaxen angewendet. Dafür implementierte ich verschiedene Versionen und kam schließlich zu einer Version mit beliebigen Backgrounds und Layers. Die Position des Hintergrunds ist nicht an den Spieler gekoppelt, sondern hat eine feste Position in der Welt. Mein Gedanke war, dass man die einzelnen Zonen weiter aufteilen kann und Variation in der Umgebung hat.

Am 28.05.2026 übernahm ich die Implementierung der Collision-Logik, da Vincent dies nach langem Versuchen nicht schaffte. Dafür übernahm er das Implementieren des Attack-Systems. Ich stieß auf mehrere Probleme. Erstens prüfte die Version von Vincent nicht den richtigen Layer. Zweitens habe ich einen anderen Ansatz probiert, den ich von StackOverflow⁴ kopiert habe, da er sinnvoll aussah. Diesen habe ich für mein Projekt angepasst, welches ebenfalls nicht funktionierte. Mithilfe von ChatGPT habe ich mehrere Ansätze und Versionen ausprobiert, die alle nicht funktionierten.

Das Prüfen des Layers funktionierte allgemein nicht, auch wenn ich auf den richtigen Typ prüfe. Ich denke, dass es an einer schlechten TileMap liegt, die nicht für das Collision-System angepasst ist.

Am 1.06.2026 habe ich die Implementierung des Kollisionssystems fortgeführt. Dabei habe ich durch meine vorherigen Versuche bemerkt, dass ich das Problem falsch angegangen bin. Ich wollte es über die Eigenschaften der einzelnen Tiles versuchen, sodass ich meine eigenen Eigenschaften, wie „solid“, benutzen kann. Es ist aber simpler, die Tile-Types, wie „solid“, im Code zu definieren und nur die GID⁵ der Objekte des CollisionLayers zu nutzen. Außerdem ist es auch effizienter und einfacher, mit einem Array zu arbeiten, in dem die Objekte mit ihrer Art gespeichert sind.

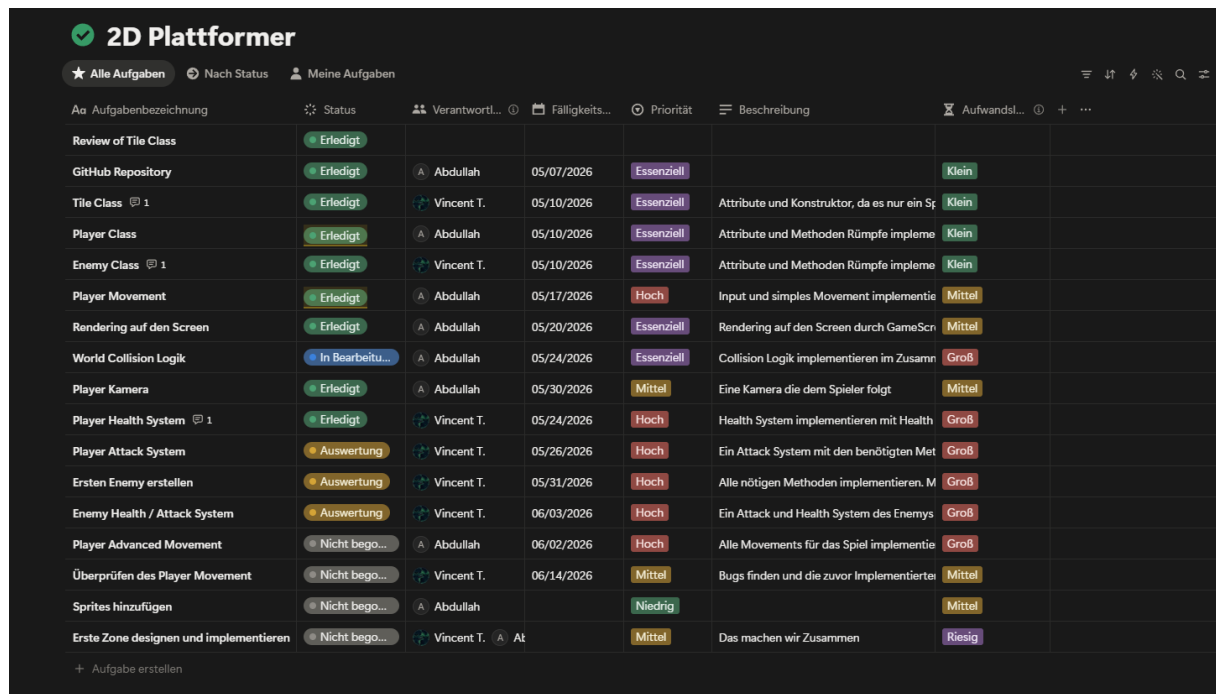
Dadurch kann ich nur die Tiles um den Spieler prüfen und die Objekte wie Tiles behandeln und den Code von Vincent zum Teil nutzen. Sein Code ist von einem TileLayer ausgegangen und nicht von einem ObjektLayer. Zudem hat er den Namen des

⁴ <https://stackoverflow.com/questions/20063281/libgdx-collision-detection-with-tiledmap>

⁵ Das ist die Globale Id der Tiles in der Map. Sie ist einzigartig.

Layers falsch geschrieben, wodurch der Code nichts machte und er die Fehler seines Codes nicht bemerkte.

Nun funktioniert die Kollision zwischen Spieler und der Welt. Zudem habe ich angefangen, ein eigenes Tilesheet für die Tiles zu nutzen, um mehr Kontrolle über das Design zu haben. Zudem ist es an unsere Physics-Engine und die Kollision angepasst. Die Integration des Gegners in dieses Kollisionssystem werde ich in den folgenden Tagen ergänzen. Unser jetziger Stand sieht folgendermaßen aus:



2D Plattformer						
Alle Aufgaben						
Aufgabenbezeichnung	Status	Verantwortl...	Fälligkeits...	Priorität	Beschreibung	Aufwandsl...
Review of Tile Class	Erfolgt					
GitHub Repository	Erfolgt	Abdullah	05/07/2026	Essenziell		Klein
Tile Class	Erfolgt	Vincent T.	05/10/2026	Essenziell	Attribute und Konstruktor, da es nur ein S...	Klein
Player Class	Erfolgt	Abdullah	05/10/2026	Essenziell	Attribute und Methoden Rumpfe impleme...	Klein
Enemy Class	Erfolgt	Vincent T.	05/10/2026	Essenziell	Attribute und Methoden Rumpfe impleme...	Klein
Player Movement	Erfolgt	Abdullah	05/17/2026	Hoch	Input und simples Movement implementie...	Mittel
Rendering auf den Screen	Erfolgt	Abdullah	05/20/2026	Essenziell	Rendering auf den Screen durch GameScn...	Mittel
World Collision Logik	In Bearbeitung...	Abdullah	05/24/2026	Essenziell	Collision Logik implementieren im Zusamm...	Groß
Player Kamera	Erfolgt	Abdullah	05/30/2026	Mittel	Eine Kamera die dem Spieler folgt	Mittel
Player Health System	Erfolgt	Vincent T.	05/24/2026	Hoch	Health System implementieren mit Health...	Groß
Player Attack System	Auswertung	Vincent T.	05/26/2026	Hoch	Ein Attack System mit den benötigten Met...	Groß
Ersten Enemy erstellen	Auswertung	Vincent T.	05/31/2026	Hoch	Alle nötigen Methoden implementieren. M...	Groß
Enemy Health / Attack System	Auswertung	Vincent T.	06/03/2026	Hoch	Ein Attack und Health System des Enemys...	Groß
Player Advanced Movement	Nicht bego...	Abdullah	06/02/2026	Hoch	Alle Movements für das Spiel implementie...	Groß
Überprüfen des Player Movement	Nicht bego...	Vincent T.	06/14/2026	Mittel	Bugs finden und die zuvor Implementierte...	Mittel
Sprites hinzufügen	Nicht bego...	Abdullah		Niedrig		Mittel
Erste Zone designen und implementieren	Nicht bego...	Vincent T.		Mittel	Das machen wir Zusammen	Riesig

Am 09.06.2026 habe ich die Kollisionslogik für den Gegner ergänzt und versucht, Wall-Jumping hinzuzufügen. Es gibt Probleme mit dem Timer, welcher das erneute Springen nach einem Wall-Jump verhindert. Der Timer ist inkonsistent, und der Spieler fällt langsam an der Wand, ohne springen zu können, obwohl man die benötigte Taste drückt. Es stellte sich heraus, dass der Timer nicht richtig auf 0 gesetzt wird, wodurch dieser immer länger wird und es sich inkonsistent anfühlt. Trotzdem konnte ich das Problem weiterhin nicht lösen. Außerdem merkte ich, dass man beim Spike direkt beim Berühren stirbt, obwohl ich einen Puffer hinzugefügt habe. Nach langer Bug-Suche mithilfe von KI habe ich keine Lösung gefunden, die funktioniert.

Am 12.06.2026 habe ich erneut versucht, das Wall-Jumping-Problem zu lösen. Ich war dieses Mal wieder nicht erfolgreich und entschied mich dazu, den Timer aufgrund von zeitlichen Gründen zu löschen. Nun kann man mehrmals an einer Wand springen und diese dadurch erklettern.

Am 19.06.2026 habe ich die Bewegung des Spielers zu einem responsiveren und Arcade-Style-Movement angepasst. Zuvor ist der Spieler aufgrund von niedrigen Werten für die Beschleunigung des Spielers gerutscht. Außerdem änderte ich die Verteilung des Lebens

des Spielers auf die angezeigten Herzen. Zudem übertrug ich diese Änderungen auf den Gegner. Darüber hinaus widmete ich mich dem Puffer für die Spike-Tiles, den ich gelöst habe. Es waren Fehler in der Semantik, die ich zur Zeit der Implementierung übersehen habe.

Ich erstellte Screen-Klassen für das Main-Menü und den Game-Over-Screen für Vincent. Zuletzt fügte ich eine Spring-Methode für den Gegner hinzu.

Am 26.06.2026 überarbeitete ich das vorhandene Angriff-System des Spielers, sodass besser für das Spiel passt. Ich konnte die vollständigen Implementierungen am Tag nicht schaffen. Meine Idee war es eine Attack Hitbox zu erstellen, diese beim Angreifen relativ zur Position des Spielers auftaucht. In der World Klasse wird dann geprüft, ob ein Gegner in dieser Hitbox ist und wird dann ggf. Schaden vom Spieler bekommen.

Am 04.07.2026 implementierte ich den Rest des Angriff-Systems. Dabei hatte ich nur das Problem, dass man als Spieler nicht weiß, ob der Gegner Schaden bekommt oder nicht. Meine Lösung wäre, den Gegner kurz rot aufleuchten zu lassen, aber ich konnte dies nicht implementieren.

Daraufhin widmete ich mich wieder dem Wall-Jump. Ich lasse das Wall-Jumping in seinem aktuellen Zustand, da die Zeit nicht mehr für die perfekte Version des Features reicht. Man kann an einer Wand jetzt durchgehend springen, was für mich als Demo reicht.

Animationen werden auch aufgrund zeitlicher Gründe ausgelassen. Zudem werden gute Sprites, Backgrounds, mehr Gegner-Typen, ein Bosskampf und eine vollständige Zone 1 mit Geheimnissen, Hinweisen auf die Story, NPCs, Cutscenes etc. aufgrund zeitlicher Gründe ausgelassen. Es wird eine simple Map zum Testen der vorhandenen Features mit simplen Gegnern geben.

Außerdem haben wir ein Main-Menü und einen Game-Over-Screen, aber ohne ein Save-Load-System mit Settings und anderen Fortschritten.

Am 06.07.2026 fügte ich die Features von Vincent zu dem Projekt hinzu und musste es in einen neuen Branch verlagern. Das Wall-Jump-System ist immer noch inkonsistent und hat Bugs. Diese liegen nicht beim Timer, sondern eher beim State- und Input-System zwischen dem Kollisionssystem für den State und dem InputHandler in der Player-Klasse.

Ich erkannte die Probleme immer wieder mithilfe von print(); und beschrieb diese ChatGPT zur Hilfe. Außerdem fügte ich meine eigenen Ideen zu dem Prompt hinzu. Das Problem habe ich erkannt, aber die Lösungen haben nicht geholfen. Morgen ist die Abgabe, und ich kann das Wall-Jump-System nicht vervollständigen, ohne noch mehrere Tage Zeit zu haben.

Ich habe allein für diesen Bug 20+ Stunden aufgewendet, und ich würde es nicht schaffen, ohne das Kollisionssystem neu umzuschreiben, da kleine Änderungen nicht funktioniert

haben. Die Demo am Ende wird eine simple Map haben, ohne Wall-Jumping und ohne ein richtiges Endziel, das erreicht werden soll. Diese Map habe ich selber erstellt.

Das Endziel der ersten Zone war eine Stadt der Gegner, die die Erde angreifen. Es sollte eine Story durch die Map erzählt werden und durch Überlebende vermittelt werden, aber dafür haben wir keine Zeit, und es wird eher ein Showcase der bisherigen Features. Wie bereits im letzten Eintrag erwähnt, werden wir keine Animationen, Musik, Sounds oder gute Sprites haben.

Der Code ist aber gut aufgebaut und kann in den meisten Fällen erweitert werden. Zudem ist er leserlich und wartbar, da Struktur und Daten getrennt wurden und jede Klasse überwiegend eine Aufgabe übernimmt. Darüber hinaus gibt es wenig Code-Duplikate, die sich aber nicht immer vermeiden lassen, da es eine Demo sein sollte und kein vollständiges Spiel. Unser Kanban-Board mit allen Features und einem theoretischen Zeitplan findet man als PDF.